

1. Introduction

Project Title:

ShopEZ – MERN Stack E-commerce Application

2. Project Overview

Purpose:

ShopEZ is a full-stack e-commerce application developed using the MERN stack (MongoDB, Express.js, React.js, and Node.js). The main goal of the application is to provide users with a smooth online shopping experience where they can browse products, add items to their cart, place orders, and manage their profiles. It also provides an admin dashboard for managing products, categories, and customer orders.

Features:

- User registration and login system.
 - Product browsing with detailed information.
 - Shopping cart functionality.
 - Secure checkout and order placement.
 - Order history and user profile management.
 - Admin dashboard for product and order management.
 - Product categories and banner management.
 - RESTful API-based communication between frontend and backend.
-

3. Architecture

Frontend:

The frontend is developed using React.js and provides an interactive and responsive user interface. It consists of reusable components and pages such as Home, Products, Authentication, Cart, User Profile, and Admin Dashboard. React Router is used for navigation and APIs are integrated to communicate with the backend.

Backend:

The backend is developed using Node.js and Express.js following the MVC (Model-View-Controller) architecture. It handles API requests, business logic, authentication, product management, cart operations, and order processing. Routes, controllers, and models are organized separately for better maintainability.

Database:

MongoDB is used as the database and Mongoose is used for schema modeling. The application contains collections for:

- Users – Stores user details and authentication information.
 - Products – Stores product details.
 - Cart – Stores products added by users.
 - Orders – Stores order information.
 - Admin – Stores categories and banner data.
-

4. Setup Instructions

Prerequisites:

Install the following software before running the project:

- Node.js (v16 or above)
- npm (v8 or above)
- MongoDB or MongoDB Atlas
- Visual Studio Code
- Git and GitHub
- Postman for API testing

Installation:

1. Clone the repository:

```
git clone <repository-url>
```

2. Navigate to the project folder:

```
cd ShopEZ
```

3. Install frontend dependencies:

```
cd client
```

```
npm install
```

4. Install backend dependencies:

```
cd server  
npm install
```

5. Configure environment variables: Create a `.env` file inside the server folder:

```
MONGO_URI=your_mongodb_connection_string
```

5. Folder Structure

Client (React Frontend):

```
client/  
├── src/  
│   ├── components/  
│   ├── pages/  
│   ├── services/  
│   ├── App.jsx  
│   └── main.jsx  
└── package.json
```

Server (Node.js Backend):

```
server/  
├── models/  
├── controllers/  
├── routes/  
├── middleware/  
├── index.js  
└── package.json
```

6. Running the Application

Start Frontend:

Navigate to the client directory:

```
npm run dev
```

Frontend runs on:

`http://localhost:5173`

Start Backend:

Navigate to the server directory:

```
nodemon index.js
```

Backend runs on:

`http://localhost:8000`

7. API Documentation

User APIs

Endpoint	Method	Description
/api/register	POST	Register a new user
/api/login	POST	Authenticate user
/api/profile	GET	Get user profile

Product APIs

Endpoint	Method	Description
/api/products	GET	Fetch all products
/api/products/:id	GET	Fetch single product
/api/products	POST	Add a new product (Admin)

Cart APIs

Endpoint	Method	Description
/api/cart	GET	Get cart items
/api/cart	POST	Add item to cart
/api/cart/:id	DELETE	Remove item from cart

Order APIs

Endpoint	Method	Description
/api/orders	POST	Create a new order

Endpoint	Method	Description
/api/orders	GET	View order history

Example Response:

```
{
  "success": true,
  "message": "Operation completed successfully"
}
```

8. Authentication

Authentication is handled using user credentials and protected routes. During login, the server verifies the user's information and provides an authentication token that is used to access authorized resources. Passwords are securely stored after encryption, and authorization checks ensure that only admins can access administrative functionalities.

9. User Interface

The application contains the following UI pages:

- Landing Page
- Product Listing Page
- Registration and Login Page
- Shopping Cart Page
- User Profile Page
- Admin Dashboard
- All Orders Page
- Product Management Page
- Add New Product Page

Screenshots can be attached in this section to showcase the interface.

10. Testing

The application is tested using:

- Postman for API testing.

- Manual testing for frontend components and user workflows.
 - Database testing to verify CRUD operations.
 - Authentication and authorization testing for secured routes.
-

11. Known Issues

- Performance may decrease when handling a large number of products without pagination.
 - Payment gateway integration is not available.
 - Additional optimization can be applied to improve loading speed.
-

12. Future Enhancements

- Integration of online payment gateways.
- Product search and advanced filtering.
- Wishlist functionality.
- Product reviews and ratings.
- Email notifications for orders.
- Real-time order tracking.
- Improved analytics for the admin dashboard.
- Better security and performance optimizations.